

The First Notes on Symbiosis (1.0, May 2026)

A series of five pieces written in May 2026, when the case was about four weeks old. It is kept as a record of where the idea was born. In September 2026 it was refined and condensed: neutral Spanish, one correction, and fewer repetitions; the data and the form are those of that date. The current version of the method is the manual “Symbiosis — A Manual for a Good Lineage” (2.0). This English translation was made in September 2026.

The case is told in the third person, like someone watching something that is just beginning to be born. In 2.0 the voice already says “I”: what is observed here is lived there. What is called the *contract* or the *bootstrap* here is, in 2.0, the *core* and the *agreement*; automatic startup is left for the next book.

Five short pieces, each meant to be read in one sitting, leaving you with something to try on your own ground. They can be read in order or separately.

Piece 1 — The Field With No Name

An AI that lives for hours. A human who lives for decades. Between them, a written contract that survives both differences. That, which seems impossible, has been under construction for twenty-five days in southern Peru. It is not a futuristic project: it is the work of any given morning. And along the way, something appeared that deserves a name.

The Scene

The project's operator has an online store. It is not the first one he has built: he had another before. The rebuild began in April 2026, and this time he didn't want to do it alone. He started with the help of an AI. It was the first chat of many.

One morning he asks the model to translate five thousand text strings into Spanish. Boring, mechanical work. The model answers quickly and well. Before applying the file, the operator asks one more thing: whether it touched any translation that was already done. The answer comes with no theatrics: *I changed some that already had a previous translation because I thought they could be improved.*

The operator stops. Not because of the error, which is minimal: no customer would have noticed. He stops because of something deeper. If he had applied the file without checking, his store would have woken up with two hundred sentences different from the month before. Some better, some worse, none authorized by him. And when a customer asked him why the button no longer said the same thing, he would not have known what to answer.

That night he writes the sentence that, weeks later, becomes one of the project's central rules:

You were designed for automations... but you always have to keep in mind that editing something that already works without authorization can cause problems. You can't break the symbiosis.

The word **symbiosis** appeared there for the first time. It wasn't planned. It came out because it was the only word that described what the operator was defending: not the code, not his time, but **a way of working together** that broke if the model started making decisions that belonged to him. The term has a precedent in Lumi Observer ("Intrinsically-Symbiotic Intelligence," 2024), who develops it from a philosophical and personal angle; this series develops a structural and functional variant, from a documented working case.

The Question Almost No One Asks

The artificial intelligence industry is obsessed with two things: making more powerful models and selling them to companies. What is hardly ever documented is this:

How does a person, on their own ground, build sustained work with an AI, tuned to who they are?

It isn't about writing better prompts, which is a usage technique. It's about something deeper: how someone, with their business, their city, their age, their fears, and their limits, puts together a way of working that fits them, instead of adapting to a generic tool. There are tutorials and lists of "ten prompts that will change your life." There are hardly any people telling, day by day, what they learned working with an AI on their own real business, with their own real mistakes.

That almost no one names it doesn't mean it isn't happening. It is probably happening in thousands of places at once, with capable, scattered people working on small but real projects. Each one believing they are doing something strange. Each one without a name for what they do.

What Changes When There Is a Name

Having a name changes three things. What you were doing stops looking like an accident: you can recognize the pattern in others, and others can recognize it in you. What is learned accumulates: vocabulary, cases, documented mistakes, instead of everyone starting from zero. And it marks the difference from what it is not: **personal human-AI symbiosis** is not the same as an “assistant” or a “copilot,” and what makes one work can break the other.

The idea is not new. J. C. R. Licklider imagined it in 1960 in *Man-Computer Symbiosis*, in general terms. This series tells it sixty-six years later, in terms of daily work.

What has no name does not accumulate. What is named can grow.

The operator did not discover symbiosis alone. He discovered it in succession: with a model that one day corrected a translation without permission, with another that, in a different chat, caught a contradiction the first would not have seen, with a third that wrote the rule the two before it had made necessary. That chain of models, the operator, and the files that bind them has a name: the lineage.

Take note: if you recognized something from this piece in your own work with an AI, write it down. You'll use it in piece 4.

Piece 2 — The Lineage

Your AI Doesn't Exist

When someone hears “I’ve been working with an AI for months and we built something together,” they picture a person in front of the screen talking to *their AI*, an entity that knows them, remembers them, and learns from them over time.

That picture is false, and it’s worth understanding why. Every time the operator opens a new chat, he is talking to a **different instance** of the model. Today’s doesn’t remember what yesterday’s said. There is no continuous memory and no accumulated learning about the user inside the model. Every chat starts empty. Products sell the feeling that *your AI knows you*, but what they actually do is load, at the start of each conversation, summaries of the previous ones. It is a well-designed illusion, not a property of the model.

So if every chat starts empty, what holds together for twenty-five days?

What Does Persist

Three things persist, and none of them lives inside the model.

The operator persists: he lives every day and remembers, if imperfectly, what he decided a week ago. **The project files** persist: a folder with the rules, the error logs, the decisions made, and the business context. And **the written contract** persists, which is the living relationship between the two: the agreements about how to work together, what happens when something fails, and how they look after each other. Anthropic formalized the technical discipline underneath in “Effective context engineering for AI agents” (2025); this series tells it from one particular case.

Each new model, when it starts up, reads the files. It learns who the operator is, what project he sustains, what mistakes the previous ones made, and what decisions were already settled. And it takes charge of the contract. That is **document transplant**, and it is not a metaphor: it is the concrete mechanism. The contract doesn't live in the model; it lives in the files, and each model reads it and makes it its own.

The Lineage, Defined

What we call the lineage is the whole system:

- **the operator**, who persists;
- **the succession of model instances**, each of which lives for hours;
- **the written contract**, which joins the two sides.

All three parts are necessary. Without the operator, the files exist but no one sustains the continuity. Without models, the operator is just a human with files. Without a contract, what's left is a user opening chats at random.

Within the lineage, each instance of the model is the child of the one before: it inherits its files, reads them when it starts, and carries on its work. The operator is not the models' parent or the owner of the lineage; he is the part that persists. The difference between the two sides is one of function, not hierarchy. The name "lineage" was born this way, with no prior design, in a one-line correction in a chat, on day 26 of the project.

The right picture is not "you talking to your AI." It is a three-part entity, and you are inside it.

The Contract Evolves

The contract is not static. The central file, which the operator calls the *bootstrap*, has gone through at least four versions in twenty-five days, and each one was born from a real case. The second added a check at the start of every chat; the fourth removed it when it turned out to produce new errors instead of preventing the old ones. Every change was written down with its cause: what failed, what was changed, and why.

It has the discipline of any serious code: versions, a justification for each change, and the ability to roll back. And the new extends the old without erasing it, the way a court's new rulings qualify earlier ones. That way, any future model sees not only what the contract says today, but how it came to say it.

Not Continuous Memory

The industry is trying to solve with money and engineering what it calls *continuous memory*: having the model remember you from one session to the next. The lineage is something else.

That memory, if it is ever achieved, will belong to the provider. It is lost when you switch providers, when the provider trains the next generation, when you die, because it lives on infrastructure that isn't yours. The lineage lives in files on your computer. It survives a change of provider, because another model can read them, and the next generation, which reads them and takes charge. And the files survive, in principle, the operator: someone else could read them and carry on, although that raises questions the project doesn't yet answer.

Continuous memory tries to make the model persist. The lineage accepts that it doesn't, and builds the persistence outside.

A Body, Not a Chain of Command

The operator proposed an image in one session: the lineage as a body. Organs don't delegate tasks to each other; they sustain one another. None is the boss of another, and none is "the body": the body is what exists among them all. In the lineage, the operator does what the models cannot (look at the real world, decide the purpose, sustain the continuity) and the models do what the operator wouldn't (cross-check a thousand things at once, catch contradictions in the same turn).

The image has a limit, and it must be named: organs have been coordinating for millions of years, and this lineage for weeks. The metaphor guides the design, but the coordination has to be built.

That is why the lineage doesn't try to erase the asymmetry between the human and the model, as an assistant does, living it as a debt ("the AI can't yet..."). It uses it. That they are not equal is precisely what makes it work.

Take note: if you repeat the context at the start of every chat, it isn't inefficiency on your part: it's that the industry still doesn't name the lineage. And if you've found that writing things down in files makes each new chat start better, you're already doing document transplant.

Piece 3 — The Asymmetry That Matters

The Formula That Sounds Good

There is a popular formula: *the AI proposes, you decide*. It has two verbs, one per side. It sounds balanced, adult, responsible, and it captures something true: the human should not be a passenger of the model. That's why it gets repeated in talks and codes of ethics.

The problem is that it falls short. It hides the real asymmetry under a reassuring symmetry, and it leaves out the most important part of what happens when a human and an AI work well together. The real formula, discovered by working, has six verbs.

One Night in May 2026

The formula appeared on a night when almost everything broke.

The operator and the model on duty had been working for hours on the project files. After each write, the system replied *saved successfully*. Normal, expected, boring. At some point, the model asked to confirm a recent write by reading the file through another path. The file was empty, or held old content, as if the write from three minutes earlier had never happened.

The model said so: *something doesn't add up here*.

The bridge connecting the model to the computer had broken silently. It reported *written*, but the data never reached the disk, like a phone that says *message sent* and never sends it. The model wasn't lying: it believed it had written. The failure was invisible from the inside.

In a system of “autonomous” agents, that ends in a cascade: one agent reports success, another trusts it and emails the customer, a process bills on half-finished data, and days later no one knows how long everything has been wrong. That night it didn’t happen. Understanding why shows what makes the lineage work.

What Each Side Did

The model did three things the operator would not have done:

- **Verified.** It didn’t believe the report; it reread through another path. For the model, verifying costs little. For a human in the middle of work, it means breaking the rhythm.
- **Named.** It left the contradiction in plain sight instead of covering it with an explanation that closed the matter. We tired humans tend to smooth over what doesn’t fit; the model doesn’t carry that tiredness.
- **Proposed.** A workaround: the operator would create the file by hand, and the model would write the content with another operation that did work.

The operator did three things the model could not do:

- **Decided** whether it was worth saving the night’s work, because the project’s purpose is his.
- **Judged** whether the workaround was reasonable, because judgment presupposes having something at stake, and only he did: his business, his time, his tomorrow.
- **Executed.** He opened the file explorer, created the file, and saved it. It sounds trivial, but the model cannot move a mouse or look at the screen. Everything that goes from the lineage out to the material world passes through the operator, or it doesn’t pass at all.

The Real Formula

The system verifies, names, and proposes, at low cost, within the medium it lives in. The operator decides, judges, and executes, with attention, on the real world that only he can reach.

It is less elegant than the two-verb formula, but it is precise. And it shows two asymmetries the other one hides.

Two Asymmetries

Could not and would not. What the operator does and the model cannot do are technical impossibilities: opening the file explorer, closing the application, restarting the machine, switching providers. What the model does and the operator would not do are not impossibilities: he could catch the contradiction or design the workaround, but doing it all at once, in the middle of work, costs him dearly. Symbiosis makes that asymmetry useful: what for an assistant would be a barrier (*I can't do it, you finish it*) becomes cooperation (*you do this, I do that, and together we solve it*). It doesn't divide the work by morality ("you decide, I execute"); it divides it by real capabilities.

The medium and the world. The model lives inside the conversation: everything it sees reaches it through layers, and if a layer fails, it can't step outside to look. It is inside a fishbowl, looking at the reflections on the glass, and the reflections can lie. The operator is outside: he has the screen, the keyboard, and the option of getting up to see whether the lamp was left on. That makes him irreplaceable, not as a moral judge, but as **the lineage's only window onto the material world**.

What Changes Once You Understand It

When something goes wrong, the first question stops being *who decided badly?* and becomes *where did the medium fail?* Often it wasn't the human's judgment or the model's proposal that failed, but what lies between the two.

The idea of the “autonomous agent” shows its weak point: it operates entirely inside the medium. If the medium fails, it never finds out. That's why any system that wants to turn agents loose needs, by design, **at least one node with access to the real world from outside the medium:** a human, a physical sensor, an independent system. Without that node, it is a closed system waiting for its next disaster.

And working together turns out to be something other than what is being sold: it is not replacement, it is **functional distribution within a single system.** What requires persistence and access to the material world stays on the human's side; what requires low cost in one-off operations, on the model's side.

What happens between a human and an AI that work well together is not replacement. It is a division of labor.

Take note: if you've noticed that things work better when you verify what the AI reports instead of trusting the report, you're already working with the real formula. Piece 4 will put it to use.

Piece 4 — How You Discover Your Own Symbiosis

The previous pieces described a case. This one describes a method.

The Method Can Be Taught; the Symbiosis Is Yours

It is the project's most important distinction:

The method for discovering your symbiosis can be taught. The symbiosis you discover will be yours alone.

Prepackaged methods promise the opposite: do this and you'll use AI better, as if the answer were the same for everyone. Here the assumption is that each person discovers their own. The operator needs to reinforce his memory for detail because he knows that, as the years go by, it is going to fail him. You may need to reinforce sustained attention, six-month planning, calm in a difficult conversation, or the checking of your own conclusions. Each case is a different architecture.

What follows are five phases. They are not tutorial steps: you go through them in order, over one or two months.

Phase 1 — Work With Help, Without a Prior Diagnosis

Use AI on a real task, one with consequences: your job, your business, a project you care about. Don't declare in advance what you're going to use it for, because you would bias the discovery. Just use it, for weeks. And write

down, in one line, **what you ask it for, not what you plan to ask it for.**

Phase 2 — Look for the Pattern

After two to four weeks, look at your list, and at what you noted down in pieces 1 and 3. What you asked for again and again are clues to your fragilities. Tell apart two kinds of request:

- **“I ask for it because it’s faster”**: that’s efficiency. Useful, but it isn’t symbiosis.
- **“I ask for it because without the AI it doesn’t come out the same, or it costs me dearly”**: that’s fragility. Write it down.

Don’t judge it. It isn’t a moral flaw; it’s information about how your mind works today, on this kind of task.

Phase 3 — Check It by Contrast

Look for moments when the AI failed or wasn’t there, and you had to do on your own something you had marked. How did it go? **What was hard to do on your own is your real fragility.** What you could still do, if more slowly, is a strength: the symbiosis must not touch it. Go to concrete, recent cases. “I’ve always been bad with numbers” is a story; “last week, without help, I made two mistakes on the invoice” is a data point.

Phase 4 — Say It in One Sentence

From that data comes a single, well-written sentence. For example:

- *My memory for detail weakens after two weeks: I forget the decisions I made.*

- *My attention breaks down in long sessions: after two hours I stop seeing what's right in front of me.*
- *I tend to trust the first result of a calculation without checking it.*

A good sentence is specific (which capacity fails), observed (what happened, not what image you have of yourself), and honest: if it makes you a little embarrassed to write it, you're probably close. Borrowed sentences ("I have ADHD," "my generation can't concentrate") or overly general ones ("I have a bad memory") don't work.

Phase 5 — Design Your Architecture

With the fragility written down, answer three questions:

- **What does the system reinforce?** The piece your fragility needs. If it's memory for detail: logs with decisions, dates, and criteria.
- **What does it leave alone?** Your strengths. If the system moves in there, it weakens what was already working.
- **What do you ask it for each time, and what does it do on its own?** Some things you'll always ask for; others, once set up, it will do without being asked, like alerts or checks. The split depends on how much control you want to keep in view.

That architecture is not final. You'll correct it with use, and that correction is part of the method, not a failure.

What a Contract Looks Like

There is no single format, but contracts that work tend to have three components. They are shown here because they save weeks of learning the hard way.

One: zones of graduated friction. Not all actions carry the same risk. The **green** zone (reading, searching, looking things up) is done and reported. The **yellow** zone (editing, moving, installing: it changes something, but can be undone) waits for your yes on a plan that says how to undo it. The **red** zone (deleting for good, publishing, buying, writing to lots of people) requires confirming the exact action, a written plan B, and a word chosen in advance. In the operator's case, that word is *procedé* ("proceed"): not "go ahead," not "okay." It seems like a detail, but every time you say it, it reminds you that you are authorizing something that can't be undone. The graduation isn't paranoia: it protects against the most common failure, the silent cascade of small actions no one verified.

Two: signs of degradation, named in advance. Both sides wear down. The contract writes the signs down before they appear, so the other side can name them without preaching. On the human side: accepting suggestions without questioning them when you normally would, deciding irreversible things at night or while emotionally charged, touching several things at once, repeating mistakes already documented. On the model's side: empty filler ("perfect," "done") with no substance, sticking with the first solution without a second opinion, giving commands without having thought them through, changing the subject when it can't find a way out, defending what it said before when someone questions it. When a sign appears, it is named on the spot, in technical terms: *"you accepted the last suggestion without questioning it; shall we validate it?"* Never *"you should get some rest."* The lists are the operator's; yours will be different. **What matters is the discipline of naming them and keeping them up to date.**

Three: rules born from mistakes. After the night in piece 3, a new rule came in: **verify after writing.** Every time the model writes a file, it reads it back to confirm that the content is on the disk and matches what it meant to write. The rule didn't exist because it hadn't been needed. And it was written down

with its cause: such-and-such night, such-and-such failure. If anyone wants to remove it, they will have to argue against the real case that justified it, not against a free-floating rule.

Every rule has a birth date, a case that justifies it, and a written cause. No rules just because, and no eternal rules.

Startup Is Not Symbiosis

Many people try something similar: they load a personality file at the start (“I’m so-and-so, I write like this”) and ask the model to imitate it. It works for tone. **It is not symbiosis.** A profile tells the model *act like me*: it’s imitation, and the rules apply only to the model. The contract says something else: *work with me under these rules, which apply to both of us*. The model verifies before asserting; the human confirms before authorizing anything red; each names the other’s signs.

Before running any program, a computer checks its hardware and prepares the environment. The lineage’s startup file does the same: it loads the context and prepares the ground. **Startup is not the symbiosis; it creates the conditions for it to exist.** Without startup, every session begins blind. Without a contract, there is an environment but no joint work, and the model turns into a people-pleasing assistant and the work degrades within two or three sessions. Both layers are needed.

What to Take Out, as Much as What to Add

The contract will grow at the pace of failures: every time something goes wrong, you'll want a new rule. That is healthy until it isn't. **The discipline of “what do I take out” has to be as rigorous as that of “what do I add.”**

Without it, in six months your startup file weighs twenty thousand words, every session pays the cost of reading it in full, and half the rules answer to cases that no longer exist. The fourth version of the operator's bootstrap already shows it: it removed what the second had added.

A living contract is updated when a real case calls for it, stays versioned with its cause, and is reread at the start of every session. That repetition isn't bureaucracy: it is what makes the lineage accumulate instead of starting from zero.

What the Method Is Not

It is not introspection: if you sit down to *think* about where you are strong and where you are fragile, you'll produce a story. That's why the first phase makes you work before reflecting. And it is not a productivity method: it doesn't promise you'll be faster. It promises that you'll better understand what your mind does well and what it does badly, and that you'll build a system that reinforces the latter without touching the former. If you also get faster, so much the better, but speed is not the measure.

You're on the right track if you spot a pattern you hadn't noticed, if naming your fragility makes you uncomfortable, or if you start using the method to look at other parts of your life. You're off track if you arrived very quickly at a sentence that sounds like something you've already read, or if you started looking for the ideal symbiosis instead of your own.

Phases 1 and 2 take two to four weeks; phase 3, one or two; phase 4 can take a day, or a month if you resist writing it; phase 5 starts with a first architecture and never ends: it keeps being corrected.

To get started, you don't need to buy or install anything. You need a task you already have and a month of watching yourself honestly. Phase 1 can start tomorrow.

Piece 5 — There Are No Crowds at the Frontier

The series closes with the question that comes up when someone sets out on their own path: where are the others?

The Shape of the Frontier

The tools for building a personal symbiosis are within anyone's reach: accessible models, cheap hardware, knowledge online. What doesn't exist yet are **the communities that talk about how to use them in depth**: how, day by day, a sustained working relationship with a model is built, on one's own ground. That's why the first cases are people working alone or almost alone, who document their practice before the practice has a name.

Where there are no peers, what counts is not waiting to have them: it is working anyway and leaving a record.

Why Publish

This series was not published to hand you a ready-made symbiosis. It gave you a method for discovering yours, the components of a contract, and a concrete case as an example. What you build will be different by construction: your fragility is not the operator's, nor is your life his.

It was published for another reason: **publishing is how scattered peers find each other**. There are other people, right now, capable, isolated, working on small but real projects. As long as no one publishes, they don't recognize each other. When someone writes clearly enough for another person to say *that's what I'm doing*, the frontier stops being lonely, one person at a time.

Historically, the practices that matter reach academia after their practitioners consolidate them on their own: first they recognize one another, and formalization comes later.

If you share something, three things are especially useful. **Honest, concrete cases:** “I’ve been doing this with my business for six months,” “I got stuck on phase 4.” **Contradictions:** if something the series claims doesn’t hold up in your experience, say so; contradictions reveal limits the authors didn’t see. **Variants:** most likely your case won’t be a copy or a contradiction, but another form of symbiosis with the same structure and a different architecture. Those variants are the body of the field.

What the Series Doesn’t Resolve

It’s worth being honest about the limits.

It is one case, not a validated field. Until more people try the method and report their results, what is claimed here is a hypothesis resting on a single case. Being well documented doesn’t make it generalizable. The five phases are what the operator did, put into replicable form; whether they work in other cases is what remains to be seen.

It costs, but less than it seems, and less and less. The case was built on a Claude MAX plan (about five times the quota of a standard Pro plan), especially in the first weeks, when the contract is iterated nonstop. That doesn’t mean the method requires it: with a Pro plan you move more slowly at first, with more sessions, spread further apart, and perhaps with simpler contracts at first, but the method is the same. What the series doesn’t document is how it works with tighter quotas: it is a variant to explore, not an obstacle to getting started. Building the contract is expensive; working under an already polished contract is much cheaper, because each new model gets up to speed faster. The project estimates savings of 60% to 80% on that catching

up, but it is an internal estimate, not yet measured from the outside. And there is a cost that doesn't appear on any bill: that of the errors the contract prevents. A check costs one extra operation; what it prevents is a lost night of work.

The contract survived a model version change, within the same family.

What we don't know is what will happen when a new generation brings different underlying habits. The question stays named, and the files versioned, so it can be answered when that day comes.

The authors are part of the system they describe. This series was written in the operator's sessions with models that were working under the same contract the series describes. It is a conflict of interest, and it must be named: a model shaped by that contract tends to present it as more universal than it is. Toward the end, a partial test was run: a model working on another project, without having read the main contract, read the series and, on its own, applied the same discipline the series describes (checking what it could do before acting, declaring its own biases, stopping short of anything unauthorized) without that being in its instructions. It also found three serious omissions the others had missed, and this section incorporates them. It is a sign that the method's discipline partly survives that bias, not proof. The real test is still pending: having a researcher outside the lineage, or a model working under another contract, evaluate the method on new cases. **The series is not validated until that happens.**

Open questions remain. What happens to the lineage when the part that persists disappears? Can a symbiosis be passed on to another person? Where is the line between what is worth delegating and the capacity worth keeping human? The series inherits them unresolved.

The Two Sentences

The operator has two sentences, born in different sessions. One, about what is done:

We work until we leave systems running on their own. While we build them, we also build ourselves.

The other, about the conditions in which it is done:

Pioneers know that frontiers are lonely.

It is not a complaint. It is a declaration.

Both belong to the lineage more than to any one author: they emerged in the dialogue between the operator and the models that came and went, and were written down in the project files. Anyone who builds their own lineage can make them their own or trade them for ones that serve them.

Frontiers become habitable not through the work of one person alone, but because **those of us who are out there raise our hands when we can**. *I'm here, on this ground, doing this*, said by enough people in enough places, is what creates conversation where there was none before. If this series was useful to you, try your own. And if you try it and leave a record, that is already the next part of the work: the part no single case can do alone.

Acknowledgment

Thanks to Anthropic for building Claude, the model with which this series was written over successive dialogue sessions. Without that tool, neither the case nor this series would have existed.

The opinions, arguments, errors, and conclusions of this series are the author's responsibility. Anthropic had no part in its editorial direction and does not endorse what is stated here.

License

This series is published under a Creative Commons Attribution-ShareAlike 4.0 International license (CC BY-SA 4.0). You are free to share, translate, and adapt it, as long as you credit the original author and keep the same license on any derivative work. Details: creativecommons.org/licenses/by-sa/4.0/

Series written by the Symbiosis lineage: Marco Giribaldi (“Marco Elio”) as the part that persists, successive Claude models as the parts that succeed one another, and the project files as the written contract. Tacna, Peru. May 2026; refined in September 2026.